Reproducing Semi-Supervised Classification with Graph Convolutional Networks

Marcos Lahoz Judith Le Roy Rémi Saout

Abstract

We reproduce and analyze the results of Kipf and Welling [2017], which introduces Graph Convolutional Networks (GCNs) for semi-supervised node classification on graphs. Our from-scratch implementation in PyTorch faithfully follows the paper's architecture, training procedure, and hyperparameters. We evaluate on the Cora, Citeseer, and Pubmed citation network benchmarks and reproduce the reported accuracy within expected variance. Additionally, we compare GCN against an MLP baseline that ignores graph structure, quantifying the contribution of neighborhood aggregation. Finally, we study the over-smoothing phenomenon by varying the network depth from 2 to 16 layers, confirming that performance degrades significantly with deeper models, and show that residual connections partially mitigate this effect.

1 Introduction

Graph-structured data is ubiquitous in applications such as social networks, citation graphs, and biological systems. A central task is *semi-supervised node classification*: given a graph where only a small fraction of nodes have labels, the goal is to predict labels for all remaining nodes.

Kipf and Welling [2017] propose a scalable approach based on *Graph Convolutional Networks* (GCNs), which learn node representations by iteratively aggregating features from local neighborhoods. The key innovation is a spectral-motivated propagation rule that is computationally efficient (linear in the number of edges) while retaining the expressive power of first-order spectral graph convolutions.

In this report, we present a from-scratch implementation of the two-layer GCN model and evaluate it on three citation network datasets. Our contributions are:

1. A faithful reproduction of the GCN results from Tables 2 and 3 of Kipf and Welling [2017], including an MLP baseline to isolate the contribution of graph structure.
2. An empirical study of the *over-smoothing* phenomenon, showing how classification accuracy degrades as network depth increases from 2 to 16 layers, and how residual connections (Appendix B of the paper) can partially mitigate this degradation.

2 Method

2.1 Graph Convolutional Networks

Consider an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with $N = |\mathcal{V}|$ nodes, adjacency matrix $A \in \{0,1\}^{N\times N}$, and node feature matrix $X \in \mathbb{R}^{N\times F}$. The GCN model learns node representations through a layer-wise propagation rule.

Renormalization trick. To incorporate self-loops and normalize by node degree, the paper defines:

$$\tilde{A} = A + I_N, \qquad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij} \tag{1}$$

where I_N is the identity matrix. The layer-wise propagation rule is then:

$$H^{(\ell+1)} = \sigma\Big(\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}\,H^{(\ell)}\,W^{(\ell)}\Big) \tag{2}$$

where $H^{(0)} = X$, $W^{(\ell)}$ is a learnable weight matrix, and σ denotes an activation function (ReLU for intermediate layers).

This symmetric normalization $\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ can be derived as a first-order approximation of localized spectral filters on graphs [Defferrard et al., 2016]. The renormalization trick avoids numerical instabilities and exploding/vanishing gradients that arise with unnormalized adjacency matrices.

2.2 Two-Layer GCN for Classification

For semi-supervised classification with C classes, the paper uses a two-layer architecture:

$$Z = \mathrm{softmax}\Big(\hat{A}\ \mathrm{ReLU}\Big(\hat{A}\,X\,W^{(0)}\Big)\,W^{(1)}\Big) \tag{3}$$

with $W^{(0)} \in \mathbb{R}^{F\times H}$ and $W^{(1)} \in \mathbb{R}^{H\times C}$, where H is the number of hidden units.

The model is trained by minimizing the cross-entropy loss over the set of labeled nodes $\mathcal{Y}_L$:

$$\mathcal{L} = -\sum_{i\in\mathcal{Y}_L}\sum_{c=1}^{C} Y_{ic}\,\ln Z_{ic} \tag{4}$$

2.3 MLP Baseline

To quantify the contribution of graph structure, we also implement a Multi-Layer Perceptron (MLP) with the same architecture ($F \to H \to C$) but without adjacency matrix multiplication:

$$Z_{\text{MLP}} = \text{softmax}\Big(\text{ReLU}(X\,W^{(0)})\,W^{(1)}\Big) \tag{5}$$

Note that the "Multi-layer perceptron" row in Table 3 of Kipf and Welling [2017] uses a single linear layer ($Z = \text{softmax}(X\Theta)$), whereas our MLP has two layers with a ReLU non-linearity (Eq. 5). This gives our baseline slightly more expressive power, which explains the higher MLP accuracy we observe on some datasets (e.g., Citeseer: 55.2% vs. 46.5% in the paper). The comparison remains valid for quantifying the contribution of graph structure, since both our GCN and MLP share the same two-layer architecture and differ only in whether neighborhood aggregation is performed.

2.4 Over-Smoothing

A known limitation of GCNs is *over-smoothing*: as the number of layers grows, repeated neighborhood aggregation causes all node representations to converge to similar values, losing discriminative power [Li et al., 2018]. After k layers of propagation with $\hat{A}$, each node's representation depends on its k-hop neighborhood. For deep networks this effectively spans the entire graph, making all representations indistinguishable.

3 Implementation Details

3.1 Architecture and Hyperparameters

We follow the exact configuration described in Section 5.2 of Kipf and Welling [2017]:

- Two-layer GCN with $H = 16$ hidden units
- Dropout rate $p = 0.5$ applied before each layer (including on input features)
- Weight initialization: Glorot uniform [Glorot and Bengio, 2010]
- Optimizer: Adam [Kingma and Ba, 2015] with learning rate $\eta = 0.01$
- L2 regularization $\lambda = 5 \times 10^{-4}$ on first-layer weights only
- Early stopping with patience of 10 epochs on validation loss
- Maximum 200 training epochs

Input features are row-normalized to unit ℓ_1-norm as specified in Section 5.2. The adjacency matrix is preprocessed once to compute $\hat{A}$ and stored as a sparse tensor.

3.2 Datasets

We evaluate on three citation network datasets from the Planetoid benchmark [Yang et al., 2016]:

Table 1: Dataset statistics.

	Cora	**Citeseer**	**Pubmed**
Nodes	2,708	3,327	19,717
Edges	5,278	4,614	44,325
Features	1,433	3,703	500
Classes	7	6	3
Train	140	120	60
Val	500	500	500
Test	1,000	1,000	1,000

The training split uses 20 labeled nodes per class (140 for Cora, 120 for Citeseer, 60 for Pubmed). The extremely low label rate (e.g., 5.2% for Cora) makes this a challenging semi-supervised setting. Citeseer requires special handling for isolated test nodes that lack connections in the graph. Our edge counts are slightly lower than those reported in Table 1 of Kipf and Welling [2017] (e.g., 5,278 vs. 5,429 for Cora) because the raw Planetoid adjacency lists contain duplicate neighbor entries that are deduplicated when building the sparse adjacency matrix.

3.3 Reproducibility Notes

We implemented the model entirely from scratch in PyTorch without relying on geometric deep learning libraries. Key implementation details that we identified through careful reading of the paper include: dropout is applied to the input feature matrix before the first graph convolution (confirmed by Appendix B); L2 regularization is applied only to the weight matrix of the first layer, excluding biases and second-layer parameters; and the Glorot initialization uses the uniform distribution $\mathcal{U}[-\sqrt{6/(f_{\text{in}} + f_{\text{out}})},\ \sqrt{6/(f_{\text{in}} + f_{\text{out}})}]$.

4 Experiments and Results

4.1 Reproduction of Paper Results

We run each model 10 times with different random seeds and report mean ± standard deviation of test accuracy. Table 2 compares our results with those reported in Tables 2 and 3 of Kipf and Welling [2017].

Table 2: Test accuracy (%) on citation networks. Paper results are from Tables 2 and 3 of Kipf and Welling [2017]. Our results are averaged over 10 runs.

Dataset	**Model**	**Ours**	**Paper**
Cora	GCN	81.6 ± 0.7	81.5
	MLP	56.9 ± 1.2	55.1
Citeseer	GCN	71.3 ± 0.5	70.3
	MLP	55.2 ± 1.4	46.5
Pubmed	GCN	78.6 ± 0.5	79.0
	MLP	72.1 ± 0.7	71.4

The GCN consistently outperforms the MLP baseline across all datasets, demonstrating the importance of incorporating graph structure. The performance gap is largest on Cora (≈26 percentage points in the paper) and smallest on Pubmed (≈8 points), which is consistent with the fact that Pubmed node features (tf-idf vectors) are already highly informative for the 3-class classification task.

Figure 1 shows typical training curves for the GCN on Cora, illustrating the convergence behavior and the point at which early stopping is triggered.

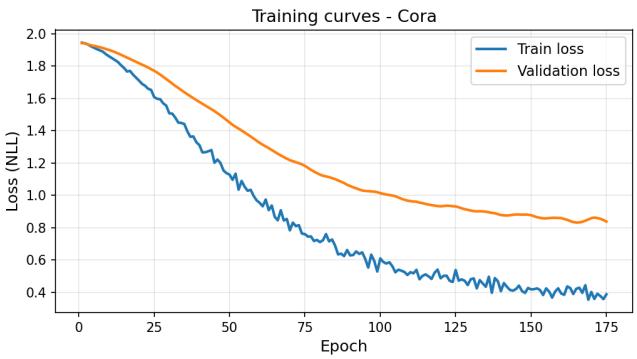

Figure 1: Training and validation loss curves for GCN on Cora. Early stopping typically triggers around epoch 135–175.

4.2 Over-Smoothing Analysis

We train GCN models with $L \in \{2, 4, 8, 16\}$ layers while keeping all other hyperparameters fixed. Results are averaged over 3 runs per configuration.

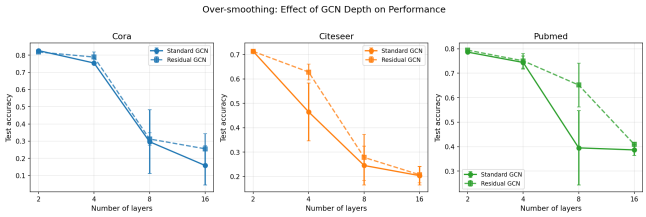

Figure 2: Test accuracy vs. number of GCN layers. Performance peaks at 2 layers and degrades significantly with increasing depth due to over-smoothing.

Table 3 and Figure 2 show that the best performance is achieved with 2 layers across all three datasets. Adding more layers leads to a sharp decline in accuracy: on Cora, accuracy drops from 82.7% at 2 layers to 15.8% at 16 layers; on Citeseer, from 71.4% to 20.4%; on Pubmed, from 78.6% to 38.7%. This confirms the over-smoothing phenomenon: after many rounds of neighborhood aggregation with $\hat{A}$, node representations become indistinguishable.

It is worth noting that the paper's Appendix B presents deeper model experiments under a different setup (5-fold cross-validation, all labels for training, 400 epochs, no early stopping). Our experiment uses the standard semi-supervised setup from Section 5 for consistency with the rest of our evaluation, which makes the over-smoothing effect more pronounced due to the smaller training set.

Table 3: Test accuracy (%) vs. number of layers (mean ± std over 3 runs). S = Standard GCN, R = Residual GCN.

	Cora		**Citeseer**		**Pubmed**	
L	S	R	S	R	S	R
2	82.7±0.5	81.8±0.6	71.4±0.6	71.3±0.5	78.6±0.2	79.4±0.2
4	75.4±0.9	78.9±2.9	46.5±11.8	62.8±3.3	74.4±2.6	75.1±3.1
8	29.6±18.5	31.2±3.6	24.5±7.9	27.8±9.4	39.5±15.2	65.2±9.0
16	15.8±11.4	25.5±8.9	20.4±3.9	20.7±3.4	38.7±2.2	40.9±0.8

4.3 Residual Connections

Following Appendix B of Kipf and Welling [2017], we implement a variant with residual (skip) connections between hidden layers:

$$H^{(\ell+1)} = \sigma\left(\hat{A}\, H^{(\ell)}\, W^{(\ell)}\right) + H^{(\ell)} \tag{6}$$

This allows the model to carry information from earlier layers, potentially mitigating the over-smoothing effect. As shown in Table 3 and Figure 2, residual connections consistently improve accuracy for deeper models. The effect is most pronounced at 4 layers, where residual connections recover a significant portion of the lost accuracy (e.g., 62.8% vs. 46.5% on Citeseer, 78.9% vs. 75.4% on Cora). On Pubmed at 8 layers, the gap is striking: 65.2% with residual vs. 39.5% without. Residual models also exhibit lower variance, suggesting more stable training. However, the two-layer model remains optimal across all datasets.

4.4 Discussion

Several observations emerge from our experiments:

Graph structure matters. The large gap between GCN and MLP confirms that neighborhood aggregation is crucial for semi-supervised classification with few labels. The MLP has access to exactly the same node features but cannot leverage graph topology, resulting in significantly lower accuracy.

Depth hurts for GCNs. Unlike deep learning in other domains (vision, NLP) where depth is beneficial, GCNs suffer from over-smoothing. Residual connections [Li et al., 2019] partially mitigate this but cannot fully overcome it, especially beyond 16 layers. Other approaches such as jumping knowledge networks [Xu et al., 2018] have been proposed to address this limitation.

Simplicity is effective. The two-layer GCN with only 16 hidden units achieves competitive results, demonstrating that a simple first-order approximation of spectral convolutions is sufficient for this task.

5 Conclusion

We presented a faithful from-scratch reproduction of the GCN model by Kipf and Welling [2017]. Our implementation reproduces the reported results on Cora, Citeseer, and Pubmed, confirming the effectiveness of graph convolutional networks for semi-supervised classification. The comparison with an MLP baseline quantifies the substantial benefit of incorporating graph structure, while the over-smoothing experiment highlights a fundamental limitation of the basic GCN architecture with increasing depth. Residual connections offer partial mitigation but do not eliminate over-smoothing, confirming that the two-layer model remains the most effective configuration for these benchmarks.

References

M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

G. Li, M. Müller, A. Thabet, and B. Ghanem. Deepgcns: Can gcns go as deep as cnns? In *International Conference on Computer Vision (ICCV)*, 2019.

Q. Li, Z. Han, and X.-M. Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *AAAI Conference on Artificial Intelligence*, 2018.

K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning (ICML)*, 2018.

Z. Yang, W. W. Cohen, and R. Salakhutdinov. Revisiting semi-supervised learning with graph embeddings. In *International Conference on Machine Learning (ICML)*, 2016.